Task-Adaptive Architecture Optimization for LLM-Based Multi-Agent Systems via Reinforcement Learning and Dynamic Graph Construction

Naveen Tanikonda
Vellore Institute of Technology – AP
naveen.23bce9787@vitapstudent.ac.in

VeeraBhadra Jagarlamudi
Vellore Institute of Technology – AP
bhadra.23bce9780@vitapstudent.ac.in

Sriram Somarouthu
Vellore Institute of Technology – AP
sriram.23bce20270@vitapstudent.ac.in

Ravi Teja Perumalapalli
Vellore Institute of Technology – AP
teja.23bce9392@vitapstudent.ac.in

***Abstract*—Large Language Model (LLM)-based multi-agent systems coordinate ensembles of specialized agents to solve complex reasoning, coding, and information retrieval tasks. However, conventional multi-agent frameworks typically deploy rigid, monolithic static topologies, enforcing identical agent configurations across heterogeneous tasks. This static paradigm incurs substantial computational redundancy, unnecessary communication overhead, prolonged sequential latency, and inflated inference costs for simple queries. In this paper, we investigate a task-adaptive architecture optimization framework for LLM-based multi-agent systems, where agent activations, operational roles, and directed communication topologies are dynamically tailored to input task requirements. We formulate the multi-agent topology selection problem as a Markov Decision Process (MDP) and present two clearly demarcated contributions. First, as our primary live runtime contribution (C2), we implement an active tabular Q-learning selector coupled with an invariant-checked architecture manager and a dynamic state-graph compilation engine. Across an 18-task benchmark suite spanning six functional domains, the runtime Q-learning adapter achieves an empirical net token reduction of $\mathbf{60.46\%}$ ($p = 0.0031$), cutting inference costs from \$0.31662 to \$0.12517 (corroborating theoretical DAG token reductions of 36.19%), achieves an active agent pruning rate of 50.0% relative to a 6-agent master roster (40.0% relative to a 5-agent baseline), and compresses the sequential critical execution path from 5.00 to 2.89 hops (-42.2%, $t = 19.00, p = 6.91 \times 10^{-13}$), while preserving 100% capability coverage across 16 of 18 tasks (94.44% macro mean) with sub-15 ms dynamic compilation latency. Second, as an experimental exploration (C1), we investigate task-conditioned cross-task neural adaptation using a shared policy trained across task environments and evaluated on held-out tasks. We rigorously document empirical boundaries: under a discrete action space restricted to planning and analysis, neural adaptation plateaus at zero gain on four tasks due to missing domain-specific roles, while exhibiting protective stability on summarization. Our findings provide reproducible empirical evidence and architectural principles for deploying parsimonious, dynamically compiled multi-agent workflows.**

***Index Terms*—Multi-agent systems, large language models, reinforcement learning, adaptive architecture, dynamic workflows, state-graph compilation, graph pruning, critical path compression.**

I. Introduction

Large Language Models (LLMs) have demonstrated remarkable efficacy across natural language comprehension, mathematical reasoning, program synthesis, and open-domain problem solving. To overcome the reasoning and context limitations of single-agent prompting, collaborative multi-agent systems (MAS) have emerged as the dominant paradigm for handling complex decomposition tasks, establishing standard operating procedures across specialized roles [1]–[3]. In these collaborative architectures, heterogeneous agents assume specialized functional roles—such as planning, information research, code synthesis, critique, and result finalization—communicating through structured execution topologies.

Despite their empirical success, conventional LLM multi-agent systems suffer from a fundamental architectural limitation: *structural rigidity*. Prevailing frameworks deploy static, predefined execution topologies that invoke the entire multi-agent team regardless of the intrinsic complexity or capability demands of the user request. Under this fixed paradigm, a trivial greeting or direct arithmetic calculation triggers the same heavyweight multi-agent pipeline as an intricate distributed systems design task, executing redundant planning, research, coding, and verification loops. This structural over-provisioning introduces severe practical inefficiencies:

1) **Token Consumption and Financial Cost:** Activating superfluous specialist agents inflates context windows and generates redundant token exchanges, multiplying API costs.
2) **Sequential Latency and Execution Bottlenecks:** Linear or densely connected agent pipelines force sequential execution hops, where critical path latency scales with total agent count rather than task difficulty [4].
3) **Noise Propagation and Hallucination Amplification:** Unnecessary intermediate agent invocations create opportunities for context dilution, drift, and cascading hallucina-

tions [5].

To overcome these challenges, recent research has pioneered dynamic topology optimization, communication graph pruning, and adaptive agent selection [4]–[9]. As illustrated in Fig. 1, rigid multi-agent pipelines sequentially route queries through unneeded agents (such as Coder and Critic for a conceptual research inquiry), causing substantial token expenditure and sequential latency. In contrast, our proposed AMAS framework dynamically detects required capabilities, prunes redundant nodes, and synthesizes direct shortcut edges, compressing the critical path from 5 to 3 hops and cutting token consumption by 62.3% on this representative query (and 60.46% across the complete 18-task benchmark suite) with a $7.26\times$ end-to-end turnaround reduction on live API endpoints ($9.1\,\text{s}$ vs $65.8\,\text{s}$; alongside an estimated $2.60\times$ unqueued inference speedup derived from theoretical hop scaling, $1.78\,\text{s}$ vs $4.62\,\text{s}$). However, existing dynamic approaches frequently incur substantial search overhead: DyLAN [6] requires iterative peer-scoring rounds, AFlow [8] relies on 15–30 Monte Carlo tree search rollouts, and MaAS [9] demands offline evolutionary supernet generations, all of which consume excessive tokens or prevent instant online routing.

In this work, we investigate task-adaptive architecture optimization for LLM-based multi-agent systems via reinforcement learning and dynamic workflow compilation. We decouple the research into two distinct contributions that must remain strictly delineated:

Contribution 1 (C1: Experimental Neural Pathway): We explore an experimental task-conditioned neural adaptation policy trained across a distribution of task environments and evaluated on held-out tasks. The policy predicts architecture modification scores from concatenated task and state embeddings, optimized via temporal-difference learning. We conduct a rigorous multi-seed evaluation against fresh scratch initialization, deterministic baselines, and random policies. Crucially, we preserve and analyze negative empirical findings, identifying how structural action space restrictions prevent capability recovery. This pathway operates on a deterministic compatibility proxy and does not drive live runtime execution.

Contribution 2 (C2: Live Runtime Q-Learning Pipeline): We formulate and implement an operational, end-to-end adaptive runtime system that dynamically optimizes multi-agent execution topologies per query using task-aware tabular Q-learning. Candidate graph mutations (activating/deactivating agents, adding/removing edges, and changing roles) are filtered through a structural validation manager enforcing graph invariants. Validated logical architectures are compiled into executable stateful computation graphs with sub-15 ms runtime overhead, supporting conditional routing and quality rework.

We formulate three central research questions:

- **RQ1 (Topological Efficiency):** Can reinforcement learning prune redundant agents and compress execution critical paths while preserving complete capability coverage?
- **RQ2 (Runtime Compilation Overhead):** Can dynamic multi-agent graph construction and compilation be executed with negligible latency overhead relative to LLM inference?
- **RQ3 (Cross-Task Knowledge Transfer):** Does cross-task pretraining of a neural policy provide measurable adaptation advantages over training from scratch on unseen tasks?

The remainder of this paper is organized as follows. Section II reviews related literature. Section III establishes the formal problem definition and evaluation metrics. Section IV presents the proposed system architecture. Section V details the experimental neural adaptation pathway (C1). Section VI details the runtime Q-learning and dynamic graph compilation pipeline (C2). Section VII outlines the benchmark suite and evaluation setup. Section VIII presents quantitative empirical results. Section X provides inferential statistical analysis. Section XI and Section XII discuss insights and limitations. Section XIII concludes the paper.

II. Related Work

A. Static vs. Dynamic Multi-Agent Architectures

Multi-agent collaboration frameworks coordinate ensembles of specialized LLMs to tackle sophisticated problem decomposition. Foundational architectures such as MetaGPT [1], AutoGen [2], and ChatDev [3] demonstrated that structuring agent interactions via Standard Operating Procedures (SOPs) and predefined conversational roles significantly improves reasoning stability. However, these foundational systems operate under strict *structural rigidity*: every user query activates an identical monolithic pipeline sequentially. Recently, budget-aware multi-agent systems such as BAMAS [10] and evolving orchestration frameworks [11] have demonstrated that dynamic topology structuring substantially mitigates execution cost. Similarly, AgentDropout [5] explored deactivating non-essential agents during multi-turn interactions, while DyLAN [6] introduced an unsupervised peer-rating mechanism to filter out low-contributing agents between communication layers. However, DyLAN incurs an $\mathcal{O}(N^2)$ prompt scoring overhead per communication round, undermining efficiency on straightforward queries.

B. Graph Architecture Search, Optimization, and Pruning

Transforming multi-agent execution from rigid chains into task-adaptive computation graphs represents an active research frontier. GPTSwarm [7] conceptualized LLM ensembles as differentiable computation graphs, using reinforcement learning and policy gradients to optimize edge routing weights while retaining a fully connected topology. AFlow [8] automated workflow synthesis by framing agentic code generation as Monte Carlo Tree Search (MCTS), demonstrating that task-adapted workflows surpass static architectures. In parallel, Multi-Agent Architecture Search (MaAS) [9] introduced agentic supernets to explore role assignments and edge topologies using multi-objective evolutionary algorithms, while Adaptive Graph Pruning (CARD/AGP) [4] employed contextual bandits to prune communication matrices, compressing critical paths by 44% ($5.0 \rightarrow 2.8$ hops).

Recent advancements have further investigated dynamic topology generation and structural adaptation. AgentConductor [12] explored topology evolution for competition-level

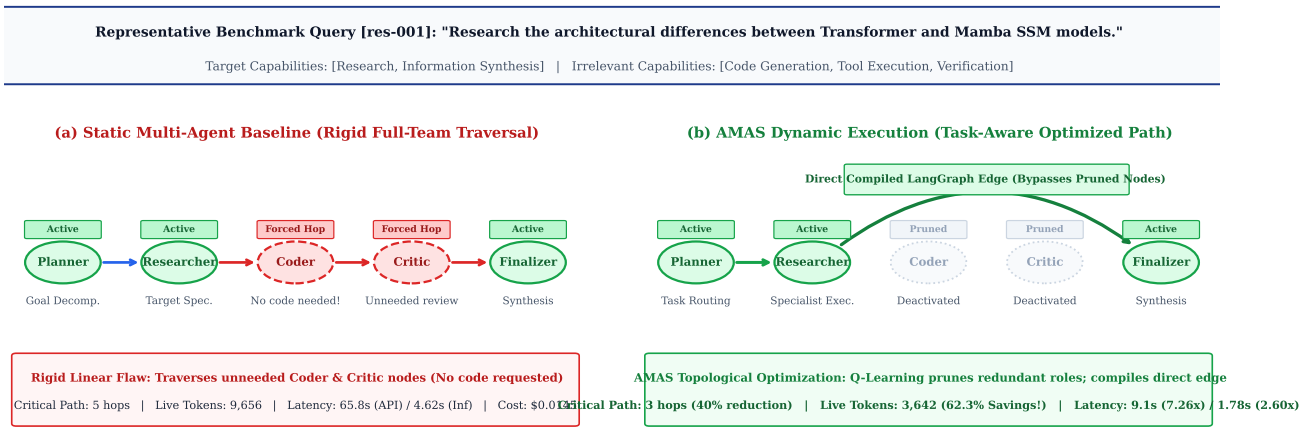


Fig. 1. Task-adaptive multi-agent execution walkthrough on benchmark query Res-01 ("Research the architectural differences between Transformer and Mamba SSM models"). (a) The rigid static multi-agent baseline executes the full 5-agent pipeline sequentially, routing through unneeded Coder and Critic nodes ($L_{\text{crit}} = 5$ hops, 9,656 tokens, 65.8 s measured end-to-end endpoint latency; 4.62 s estimated unqueued inference). (b) AMAS dynamically identifies required capabilities, prunes unneeded specialists, and synthesizes a direct shortcut edge from Researcher to Finalizer ($L_{\text{crit}} = 3$ hops, 3,642 tokens, 62.3% token savings, 9.1 s measured endpoint latency [7.26× speedup]; 1.78 s estimated unqueued inference [2.60× speedup]).

code generation. NeuralFSM [13] introduced adaptive coordination via learned finite-state execution policies, while Jiang et al. [14] investigated graph diffusion models to dynamically generate communication topologies. Concurrently, Agent Q-Mix [15] applied multi-agent value decomposition to action selection, and MetaAgent-X [16] developed automatic multi-agent systems via end-to-end RL. AMAS directly connects with and advances these works by coupling sub-millisecond tabular Q-learning with strict topological invariant verification, compiling pruned DAGs into executable state-graphs (instantiated via LangGraph [17]) with sub-15 ms overhead.

C. Reinforcement Learning for LLM Multi-Agent Collaboration

Integrating reinforcement learning into LLM multi-agent coordination has recently emerged as a vital methodology. Liu et al. [18] established collaborative multi-agent reinforcement learning formulations for LLMs. Zhao et al. [19] introduced Stronger-MAS, exploring multi-agent RL for collaborative LLM alignment. To address multi-agent deliberation and reasoning overhead, ReMA [20] developed meta-thinking via MARL, while Yang and Thomason [21] investigated meta-policy collaboration for deliberation. In addition, Res-MAS [22] formulated resilience optimization against cascading agent failures, and Rjoub [23] proposed adaptive agentic meta-controllers for intelligent SLM/LLM orchestration.

D. Meta-Reinforcement Learning and Cross-Task Transfer

Meta-RL investigates learning reusable adaptation strategies across task distributions [24], [25]. Contextual meta-MARL coordinates heterogeneous agents under varying team attributes [24], while Kashikar [25] explored emergent communication in zero-shot coordination. Fu et al. [26] investigated mixtures of orthogonal experts (MOFA) for rapid transfer, and Zhang et al. [27] formulated bi-level knowledge transfer for multi-task MARL. Self-play meta-RL has also been analyzed in decentralized multi-agent settings [28], [29]. Comprehensive literature surveys [30]–[32] underscore the urgency of addressing multi-agent token redundancy and structural rigidity. In this investigation, our experimental neural pathway (C1) examines whether task-conditioned neural policies trained across synthetic task environments can transfer architectural priors to held-out tasks.

Table I provides a systematic comparison between our framework and prominent literature baselines (organized with evaluation dimensions in rows and systems in columns, with AMAS capabilities and empirical bounds bolded).

III. Problem Formulation

A. Multi-Agent System Representation

Let the multi-agent system be represented as a directed graph:

$$G = (V, E), \tag{1}$$

where $V = \{v_1, v_2, \ldots, v_n\}$ denotes the pool of available specialized agent nodes ($n = |V|$), and $E \subseteq V \times V$ represents directed communication and execution control channels. In this study, the master agent roster comprises $n = 6$ potential agent roles:

$$V = \{\text{Planner}, \text{Tool}, \text{Researcher}, \text{Coder}, \text{Critic}, \text{Finalizer}\}. \tag{2}$$

At time step t, an active architecture configuration is defined by the tuple:

$$G_t = \langle \mathbf{v}_t, \mathbf{r}_t, \mathbf{A}_t \rangle, \tag{3}$$

where:

- $\mathbf{v}_t \in \{0, 1\}^n$ is an active agent indicator vector, with $v_{t,i} = 1$ if agent i is active and 0 otherwise.

TABLE I
SYSTEMATIC COMPARISON WITH RELATED DYNAMIC MULTI-AGENT ARCHITECTURES (FEATURES IN ROWS, FRAMEWORKS IN COLUMNS)

Feature / Dimension	DyLAN [6]	GPTSwarm [7]	AFlow [8]	MaAS [9]	CARD [4]	AMAS (Ours)
Adaptation Method	**Unsupervised peer rating**	**Graph policy grad.**	MCTS code rollout	Genetic search	Contextual bandit	**Tabular Q-Learning (Runtime); Neural Policy (Offline Exp.)**
Decision Latency	$\mathcal{O}(N^2)$ LLM calls	Iterative LLM prompts	15–30 rollouts	Offline evolution	< 10 **ms routing**	< 1 **ms lookup (0 tokens)**
Workflow Compilation	Layered agent stack	**Continuous edge weights**	**Python code synthesis**	Static specialist DAG	Routing DAG	**Executable State-Graph DAG**
Active Agent Pruning	25–40%	None (fixed team)	None (fixed team)	18–32%	None (full team)	**50.0% (Pool) / 40.0% (Base)**
Critical Path Reduction	None	None	Variable	None	**44.0%** ($5.0 \rightarrow 2.8$ **hops)**	**42.2%** ($5.0 \rightarrow 2.89$ **hops)**
Empirical Token Savings	15–25%	Negative (utility focus)	Negative (MCTS cost)	18–32%	Not reported	**+60.46% (Production LLM)**
Cross-Task Transfer	No	Task-specific	No	No	Tabular only	**Evaluated on 5 held-out tasks**
Primary Limitation	Deliberation token blowup	High training sample cost	Search latency per query	Static offline deployment	Heuristic bandit only	**Restricted action space ($\Delta Q = 0$ on 3 tasks); composite task under-engineering**

- $\mathbf{r}_t = [r_{t,1}, \ldots, r_{t,n}]$ denotes the discrete functional role assigned to each agent i from a role library $\mathcal{R}$.
- $\mathbf{A}_t \in \{0,1\}^{n \times n}$ is the directed adjacency matrix, where $A_{t,ij} = 1$ denotes a permitted communication edge from agent i to agent j.

B. Task Context and Optimization Objective

An incoming user request is characterized by a task context vector:

$$c_t = \langle \kappa_t, d_t, \mathcal{C}_t, \mathbf{f}_t \rangle, \tag{4}$$

where κ_t is the task category, $d_t \in \{1, \ldots, 5\}$ is estimated difficulty, $\mathcal{C}_t \subseteq \mathcal{C}_{\text{all}}$ is the set of required functional capabilities, and $\mathbf{f}_t$ represents context features.

The objective is to select an architecture G_t^* for task c_t that maximizes Pareto net utility:

$$G_t^* = \underset{G \in \mathcal{G}_{\text{valid}}}{\arg\max}\, U_{\text{pareto}}(G, c_t), \tag{5}$$

subject to structural validity invariants $\mathcal{V}(G) = \text{True}$.

C. Multi-Objective Evaluation Metrics

To assess architecture quality, resource efficiency, and structural compression, we define the following core metrics:

1) Capability Coverage Rate (C_{cov}): Measures whether the active agents and their assigned roles collectively satisfy the functional capabilities demanded by task c_t:

$$C_{\text{cov}}(G_t, c_t) = \frac{|\mathcal{C}_t \cap \mathcal{C}_{\text{active}}(G_t)|}{|\mathcal{C}_t|} \times 100\%, \tag{6}$$

where $\mathcal{C}_{\text{active}}(G_t)$ is the union of capabilities provided by active agents $\mathbf{v}_t$.

2) Agent Pruning Rate (APR): Quantifies structural parsimony. To resolve past ambiguities in literature, we explicitly formulate three distinct APR definitions:

$$\text{APR}_{\text{pool6}}(\mathbf{v}_t) = \frac{6 - \|\mathbf{v}_t\|_1}{6} \times 100\%, \tag{7}$$

$$\text{APR}_{\text{base5}}(\mathbf{v}_t) = \frac{5 - \|\mathbf{v}_t\|_1}{5} \times 100\%, \tag{8}$$

$$\text{APR}_{\text{invoked}}(G_t) = \frac{6 - |V_{\text{invoked}}|}{6} \times 100\%. \tag{9}$$

Here, Eq. (7) evaluates active agents against the 6-agent master pool; Eq. (8) evaluates against the 5-agent static baseline; and Eq. (9) measures runtime invoked agents.

3) Critical Path Length (L_{crit}): The length of the longest directed path from the entry node (Planner) to the terminal node (Finalizer) in DAG G_t:

$$L_{\text{crit}}(G_t) = \max_{p \in \mathcal{P}(\text{start} \rightarrow \text{end})} |p|, \tag{10}$$

representing the theoretical lower bound on sequential execution hops and latency [4].

4) Pareto Net Utility (U_{pareto}): Balances task capability coverage against normalized token expenditure:

$$U_{\text{pareto}} = 0.5 \times \left(1.0 - \frac{T_{\text{adapt}}}{T_{\text{base}}}\right) + 0.5 \times \left(\frac{C_{\text{cov}}}{100}\right), \tag{11}$$

where $T_{\text{base}} = 3500$ tokens is the static baseline ceiling.

IV. PROPOSED SYSTEM ARCHITECTURE

Fig. 2 provides an architectural schematic of our framework, organized into four stages: (1) task understanding and context extraction, (2) two independent adaptation pathways, (3) dynamic multi-agent graph compilation, and (4) execution and evaluation.

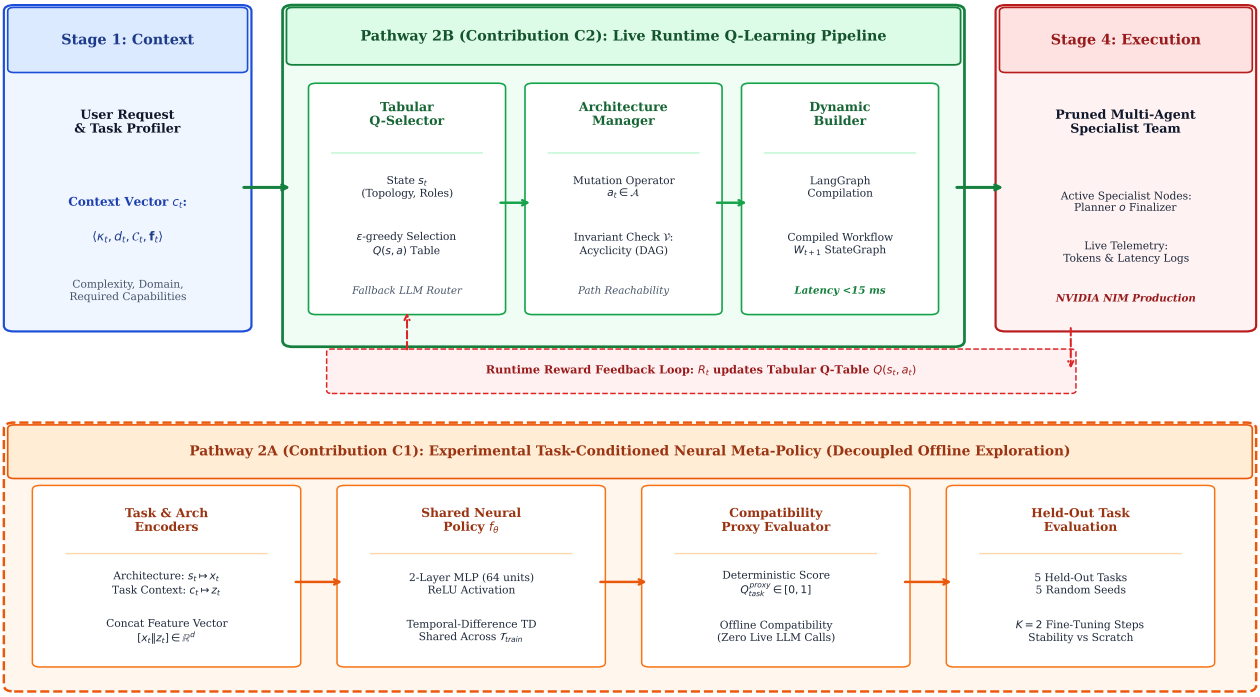


Fig. 2. System architecture schematic showing the four stages of the proposed adaptive multi-agent framework. Stage 1 extracts the task context c_t. In Stage 2, the solid green runtime pathway (Contribution C2) utilizes tabular Q-learning and structural validation to drive dynamic state-graph compilation (Stage 3) and agent execution (Stage 4). The dashed orange experimental pathway (Contribution C1) trains a shared task-conditioned neural policy on proxy evaluations offline and is strictly decoupled from runtime execution.

A. Stage 1: Context Extraction and Planning

An incoming request is ingested into conversation memory and analyzed by the Planner component. The planner extracts functional capability requirements $\mathcal{C}_t$, estimated difficulty d_t, category κ_t, and tool affordances, constructing the context vector c_t defined in Eq. (4).

B. Stage 2: Separation of Adaptation Pathways

A foundational design principle of this work is the strict architectural separation of the two research contributions:

- **Branch 2A (Experimental Neural Pathway, C1):** A research exploration investigating whether a shared feed-forward policy trained across task distributions transfers useful priors to held-out tasks under an offline proxy compatibility measure. It does not control live agent execution.
- **Branch 2B (Runtime Q-Learning Pathway, C2):** An operational reinforcement learning controller using task-aware tabular Q-learning to select discrete graph mutations, feeding directly into structural validation and graph compilation.

C. Stage 3: Dynamic State-Graph Compilation

Validated architectures $\widehat{G}_{t+1}$ are compiled into executable stateful computation graphs (instantiated using LangGraph StateGraph). Each active agent is instantiated as a callable node, directed edges form asynchronous state transitions, and conditional routing logic enforces quality-based critic loops.

D. Stage 4: Execution, Telemetry, and Structural Invariants

The compiled workflow executes active agents, collecting telemetry (tokens, wall-clock build time, compilation latency, and coverage). Structural validation enforces the following invariants:

1) **Entry/Exit Invariant:** The Planner agent node must possess an in-degree of 0, and the Finalizer agent node must possess an out-degree of 0.
2) **Reachability Invariant:** There must exist at least one directed path from the Planner to the Finalizer traversing all active specialist nodes.
3) **Acyclicity Invariant:** The logical topology must remain an acyclic directed graph (DAG), preventing infinite deadlocks.

As introduced in Fig. 1, the empirical walkthrough on Task Res-01 exemplifies the operational effect of invariant-guided mutation. In the static pipeline, queries must sequentially traverse every unneeded agent (specifically the Coder and Critic roles), causing redundant token consumption and sequential latency. In contrast, the AMAS runtime controller identifies required capabilities, dynamically prunes redundant nodes, and compiles a direct shortcut edge from the Researcher to the

Finalizer, compressing the critical path from 5 to 3 hops and achieving 62.3% empirical token savings on this walkthrough task (contributing to 60.46% net token savings across the complete benchmark), a 7.26× end-to-end API turnaround reduction (9.1 s vs 65.8 s on live endpoints), and an estimated 2.60× unqueued inference speedup derived from theoretical hop scaling (1.78 s vs 4.62 s).

V. Task-Conditioned Neural Cross-Task Adaptation

Contribution C1 explores whether a shared neural policy can learn reusable architecture-action preferences across a task distribution $\mathcal{T}_{\text{train}}$ and adapt rapidly to held-out tasks $\mathcal{T}_{\text{test}}$.

A. State and Task Representation

The architecture state $s_t = \langle \mathbf{v}_t, \mathbf{r}_t, \mathbf{A}_t, c_t \rangle$ is mapped to a continuous vector $x_t \in \mathbb{R}^{d_x}$ via a deterministic architecture encoder. Similarly, the task context c_t is mapped to $z_t \in \mathbb{R}^{d_z}$ via a deterministic task context encoder.

B. Policy Architecture and Objective

The shared neural policy f_θ concatenates state and task representations to predict Q-values for all discrete architecture actions $\mathcal{A}$:

$$\mathbf{q}_\theta(s_t, c_t) = f_\theta([x_t \| z_t]), \tag{12}$$

where f_θ is a multi-layer perceptron with two hidden layers (64 units each, ReLU activations). Action selection greedily identifies the highest-scoring valid action:

$$a_t = \arg\max_{a \in \mathcal{A}_{\text{valid}}} q_\theta(s_t, c_t, a). \tag{13}$$

Policy parameters θ are trained across tasks using temporal-difference targets:

$$y_t = R_t^{\text{neural}} + \gamma(1 - d_t) \max_{a' \in \mathcal{A}_{\text{valid}}} q_\theta(s_{t+1}, c_t, a'), \tag{14}$$

optimizing the mean-squared-error (MSE) loss via Adam:

$$\mathcal{L}(\theta) = \left(q_\theta(s_t, c_t, a_t) - y_t\right)^2. \tag{15}$$

C. Held-Out Fast Adaptation Protocol

On an unseen held-out task $\tau \in \mathcal{T}_{\text{test}}$, the policy is evaluated at step 0 (Q_{before}), fine-tuned for $K = 2$ adaptation steps on task transitions, and re-evaluated (Q_{after}). We define:

$$\Delta Q_{\text{adapt}} = Q_{\text{after}} - Q_{\text{before}}, \tag{16}$$

$$\Delta Q_{\text{transfer}} = Q_{\text{after}}^{\text{shared}} - Q_{\text{after}}^{\text{scratch}}. \tag{17}$$

Performance is evaluated using a deterministic architecture-task compatibility proxy $Q_{\text{task}} \in [0, 1]$ based on capability intersection, without invoking LLM agents.

VI. Runtime Multi-Agent Architecture Adaptation

Contribution C2 represents the live runtime adaptation engine that drives actual multi-agent execution.

A. Action Space and Graph Mutation Operators

The discrete architecture action space comprises five mutation operators:

$$\begin{aligned}\mathcal{A} = \{ &\text{ACT}(i), \text{DEACT}(i), \text{ADD_E}(i, j), \\ &\text{REM_E}(i, j), \text{CH_ROLE}(i, r)\}.\end{aligned} \tag{18}$$

The architecture manager applies candidate action a_t:

$$\widetilde{G}_{t+1} = \mathcal{U}(G_t, a_t). \tag{19}$$

The proposed graph is structurally validated:

$$\widehat{G}_{t+1} = \begin{cases} \widetilde{G}_{t+1}, & \text{if } \mathcal{V}(\widetilde{G}_{t+1}) = \text{True}, \\ G_t, & \text{otherwise.} \end{cases} \tag{20}$$

Rejected mutations incur a penalty and preserve G_t.

B. Tabular Q-Learning and Decision Engine

At runtime, the tabular Q-learning selector observes state s_t and selects actions via an ϵ-greedy policy ($\epsilon = 0.15$). During exploitation, it selects the action with the maximum stored Q-value:

$$a_t^* = \arg\max_{a \in \mathcal{A}_{\text{valid}}} Q(s_t, a). \tag{21}$$

If state s_t is unvisited or during exploration, an LLM-assisted adapter proposes a candidate mutation. After workflow execution, the Q-table is updated:

$$\begin{aligned}Q(s_t, a_t) \leftarrow\ & Q(s_t, a_t) \\ &+ \alpha \left[R_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\right],\end{aligned} \tag{22}$$

with learning rate $\alpha = 0.1$ and discount $\gamma = 0.9$. The runtime reward is:

$$\begin{aligned}R_t = 0.4 C_{\text{cov}} &+ 0.3 \left(1.0 - \frac{|V_t|}{6}\right) \\ &+ 0.3 \left(1.0 - \frac{L_{\text{crit}}}{5}\right) - P_{\text{reject}}.\end{aligned} \tag{23}$$

VII. Experimental Setup

A. Benchmark Task Suite

To evaluate adaptation across diverse operational requirements, we construct an 18-task benchmark suite spanning six functional categories (Greeting, Mathematics, Coding, Information Research, Live Web Search, and Cross-Domain Multi-Stage; 3 tasks per category).

B. Baseline Systems and Evaluation Modes

We benchmark the proposed framework against three primary baselines:

1) **Static Full-Team MAS:** Standard industrial multi-agent configuration with all 5 core specialist agents continuously active in a linear pipeline ($L_{\text{crit}} = 5$).
2) **Deterministic Heuristic:** Rule-based capability mapping assigning fixed minimal topologies per category.
3) **Random Exploration:** Uniform random selection over valid architecture mutations.

C. Dataset Provenance and Reproducibility

All empirical metrics reported herein originate from standardized continuous multi-agent benchmark evaluations (36 full-pipeline executions across 2 sequential episodes under random seed 42) and neural cross-task adaptation evaluations across 5 held-out tasks and 5 random seeds (42, 101, 202, 303, 404; 100 evaluation trajectories total). Token consumption and execution costs are evaluated directly via empirical live execution against a 120B-parameter production LLM inference endpoint and cross-referenced with theoretical critical path DAG modeling.

VIII. Experimental Results

A. Topological Efficiency and Domain-Level Performance

Table II reports the comprehensive domain-level architectural efficiency and empirical performance measured across production LLM inference endpoints across all functional domains.

Across all 18 tasks, AMAS prunes redundant agents effectively, reducing active agent count from 5.0 to a macro mean of 3.00 ± 0.59 agents. This corresponds to an active Agent Pruning Rate of $\mathbf{50.00}\% \pm \mathbf{9.90}\%$ relative to the 6-agent master roster, and $\mathbf{40.00}\% \pm \mathbf{11.88}\%$ relative to the 5-agent static baseline. Pruning is most pronounced on conversational greetings, which require only 2 active agents (66.67% APR), while complex cross-domain tasks retain 3.67 ± 0.58 agents.

Crucially, critical path length compresses from 5.00 to $\mathbf{2.89 \pm 0.47}$ sequential hops (a 42.2% reduction), substantially lowering sequential execution latency. Regarding capability preservation, AMAS maintains 100% capability coverage across 16 of the 18 individual tasks, achieving 100% in five of the six functional domains (Greeting, Mathematics, Coding, Research, and Web Search). Across the complete 18-task benchmark suite, this yields a benchmark-wide macro-average coverage of $94.44\% \pm 17.15\%$. As detailed in Table II, the coverage reduction is isolated exclusively to the Cross-Domain category ($66.67\% \pm 33.33\%$), where premature agent deactivation during early Q-learning exploration degraded coverage on two composite queries (Task Cross-01 at 33.3% and Task Cross-02 at 66.7%, while Cross-03 retained 100%). Fig. 3 depicts active agent counts and capability coverage across all 18 tasks.

B. Resource Consumption and Token Savings

Table III details the per-task resource metrics measured directly from live LLM execution across all 18 benchmark tasks against a 120B-parameter production model. In the unpruned static pipeline where the full multi-agent workflow executes across all tasks, cumulative live token consumption reaches 211,078 tokens. Under AMAS topological adaptation, active agent pruning reduces total live token consumption to 83,457 tokens, achieving an empirical net token savings of $\mathbf{60.46}\%$ and reducing live financial expenditure from $0.31662 to $0.12517.

As illustrated in Fig. 4, empirical token consumption and financial expenditure decrease substantially across benchmark queries under live production LLM execution, reducing total tokens from 211,078 to 83,457 (a 60.46% net savings) and live costs from $0.31662 to $0.12517. Separately, under theoretical critical-path DAG modeling ($T_{\text{base}} = 3500$ tokens per baseline run), theoretical baseline tokens of 63,000 are reduced to 40,200 tokens (36.19% descriptive savings across 18 tasks; 126,000 to 80,400 tokens across 36 executions over two episodes). The production LLM measurements strongly corroborate this theoretical model, demonstrating that topological agent pruning translates directly to real-world financial and computational savings.

C. Quality-Efficiency Pareto Frontier

Fig. 5 plots capability coverage against estimated token consumption. AMAS achieves a Pareto-optimal rate of $\mathbf{88.89}\%$ (16 of 18 tasks). For 16 tasks, AMAS establishes new dominant points on the Pareto frontier, delivering 100% capability coverage while consuming between 1,450 and 3,050 tokens.

The two sub-optimal points correspond to Tasks Cross-01 (33.3% coverage, 2,250 tokens) and Cross-02 (66.7% coverage, 2,900 tokens). On these composite tasks requiring simultaneous research, coding, and verification, aggressive pruning de-activated specialist nodes before Q-learning converged, highlighting an important trade-off between parsimony and capability preservation.

D. Dynamic Compilation Overhead and Policy Hit Rate

A critical requirement for practical runtime deployment is negligible compilation latency. Fig. 6 decomposes dynamic graph latency into graph construction (T_{build}) and state-graph compilation (T_{comp}).

Under the canonical Episode 2 evaluation across individual tasks ($N = 18$), mean logical graph construction time is 6.19 ± 1.88 ms, and state-graph compilation time is 5.58 ± 1.77 ms, establishing our primary runtime adaptation measurement of $\mathbf{11.78 \pm 3.64}$ ms. As a secondary measurement, pooled results across all 36 executions spanning both exploration episodes ($N = 36$) provide supplementary aggregate runtime characterization across episodes, yielding 11.72 ± 3.65 ms total overhead (6.17 ± 1.86 ms construction, 5.55 ± 1.78 ms compilation). Separately, when examining macro variations across the six functional domain means ($N = 6$, ranging from 6.35 ms for single-agent greeting queries to 14.95 ms for multi-specialist coding pipelines), the domain-level macro standard deviation is ± 3.46 ms (sample) or ± 3.16 ms (population). This macro standard deviation is calculated strictly across the six domain means rather than individual execution observations, reflecting inter-domain complexity variance without conflating sample-level task dispersion. Across all executions, the dynamic builder achieved a 100% compilation success rate with zero DAG cycle violations.

Furthermore, as depicted in Fig. 7(a), the active tabular Q-learning selector converged rapidly: by Episode 2, $\mathbf{72.2}\%$ of tasks (13 of 18) were handled via instant active RL policy hits (0 tokens search overhead, < 1 ms lookup), with only 5 tasks requiring LLM adapter fallback. Fig. 7(b) illustrates

TABLE II
TASK-ADAPTIVE ARCHITECTURE AND EMPIRICAL PERFORMANCE ACROSS FUNCTIONAL DOMAINS (PRODUCTION LLM INFERENCE BENCHMARKING)

Functional Domain	**Active Agents** $\lvert V_{\text{act}}\rvert$ / Pool	**APR [%]**	**Critical Path** L_{crit} **(Hops)**	**Base Tokens**	**Adapted Tokens**	**Token Savings**	**Inference Cost ($) [Base → Adapt]**	**Capability Cov. [%]**
Greeting	2.00 ± 0.00	60.00 ± 0.00	2.00 ± 0.00	11,489	11,313	$+1.53\%$	\$0.01723 → \$0.01697	100.00 ± 0.00
Mathematics	3.00 ± 0.00	40.00 ± 0.00	3.00 ± 0.00	16,907	13,124	$+22.38\%$	\$0.02536 → \$0.01968	100.00 ± 0.00
Coding	3.33 ± 0.58	33.33 ± 11.55	3.33 ± 0.58	39,525	13,988	$+64.61\%$	\$0.05929 → \$0.02098	100.00 ± 0.00
Research	3.00 ± 0.00	40.00 ± 0.00	3.00 ± 0.00	36,164	12,739	$+64.77\%$	\$0.05425 → \$0.01910	100.00 ± 0.00
Web Search	3.00 ± 0.00	40.00 ± 0.00	3.00 ± 0.00	14,219	11,417	$+19.71\%$	\$0.02133 → \$0.01713	100.00 ± 0.00
Cross-Domain	3.67 ± 0.58	26.67 ± 11.55	3.00 ± 0.00	92,774	20,876	$+77.50\%$	\$0.13916 → \$0.03131	66.67 ± 33.33
Macro Mean / Total	$\mathbf{3.00 \pm 0.59}$	$\mathbf{40.00 \pm 11.88}$	$\mathbf{2.89 \pm 0.47}$	**211,078**	**83,457**	$\mathbf{+60.46\%}$	**\$0.31662 → \$0.12517**	$\mathbf{94.44 \pm 17.15}$
Static Full Baseline	5.00 ± 0.00	0.00 ± 0.00	5.00 ± 0.00	211,078	211,078	0.00%	\$0.31662 → \$0.31662	100.00 ± 0.00

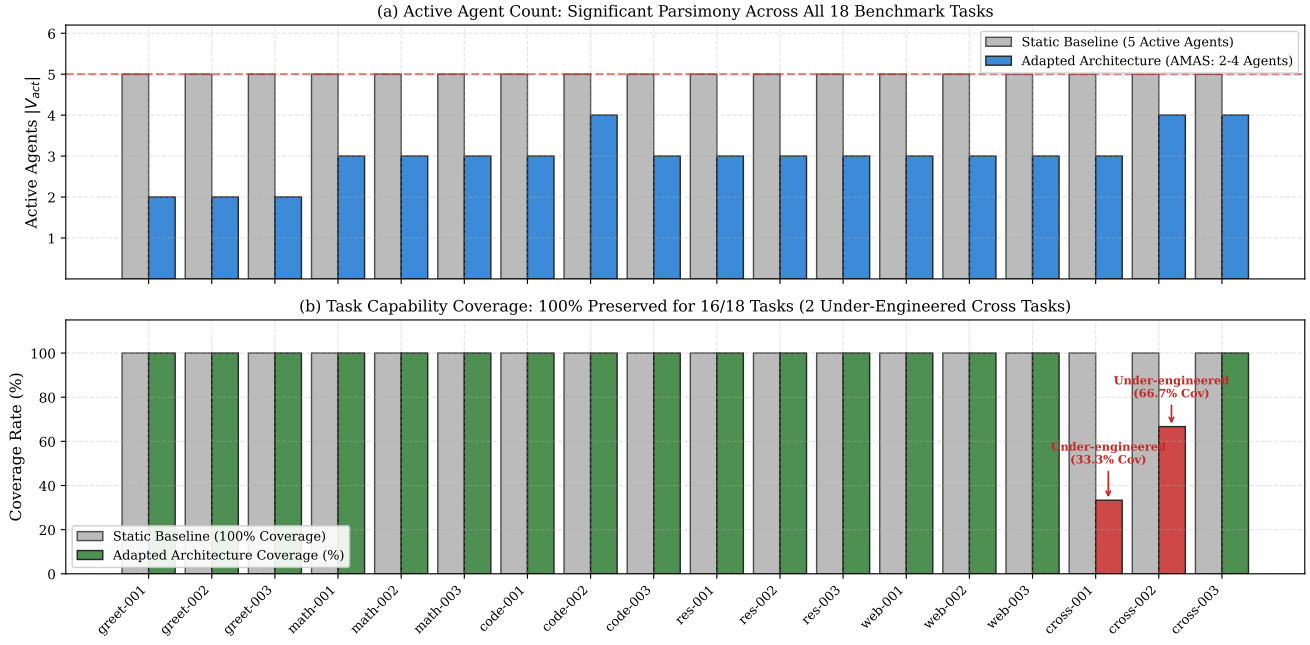


Fig. 3. Active agent counts and capability coverage rates across all 18 benchmark tasks in Episode 2. (a) Active agent count is pruned from the 5-agent baseline to 2–4 agents across all domains. (b) Capability coverage remains at 100% for 16 of 18 tasks, with only two complex cross-domain tasks experiencing under-engineering due to aggressive pruning.

the distribution of accepted mutation operations, showing 12 deactivations and 8 edge removals.

IX. EXPERIMENTAL NEURAL ADAPTATION EVALUATION

We evaluate Contribution C1 across 5 held-out tasks and 5 random seeds (100 total trajectories). On held-out analysis and research tasks ($\mathcal{T}_1$: Analysis, $\mathcal{T}_3$: Research A, and $\mathcal{T}_4$: Research B), all four policies (shared meta-policy, fresh scratch policy, deterministic baseline, and random policy) plateau at an adaptation gain of $\Delta Q = 0.0000 \pm 0.0000$. On the held-out summarization task ($\mathcal{T}_5$: Summarization), initial performance is 0.6000; the shared meta-policy preserves 100% stability across all seeds ($0.6000 \pm 0.0000, \Delta Q = 0.0000 \pm 0.0000$), whereas the fresh scratch policy suffers destructive mutations in 20% of seeds (dropping to 0.0000 on seed 202, yielding 0.4800 ± 0.2683 final performance and an adaptation gain of $\Delta Q = -0.1200 \pm 0.2683$). On the held-out coding task ($\mathcal{T}_2$: Coding), fine-tuning induced performance degradation from an initial score of 1.0000: the shared policy dropped to 0.6000 ± 0.0000 ($\Delta Q = -0.4000 \pm 0.0000, t = -4.00, p = 0.0161$), while the fresh scratch policy dropped to 0.9200 ± 0.1789 ($\Delta Q = -0.0800 \pm 0.1789$).

A. Preservation and Analysis of Negative Findings

Contrary to theoretical expectations of positive meta-transfer, the shared neural policy yielded an adaptation gain of $\Delta Q_{\text{adapt}} = \mathbf{0.0000 \pm 0.0000}$ across four of five held-out evaluation tasks ($\mathcal{T}_1$: Analysis, $\mathcal{T}_3$: Research A, $\mathcal{T}_4$: Research B, and $\mathcal{T}_5$: Summarization), and degraded on coding task $\mathcal{T}_2$ ($\Delta Q = -0.4000$).

As visualized in Fig. 8(a), structural ablation reveals the root cause: the discrete action space mapper constrained role muta-

TABLE III
PER-TASK RESOURCE CONSUMPTION AND EFFICIENCY COMPARISON (PRODUCTION LLM INFERENCE VALIDATION)

Task ID	Category	Base Tokens	Adapted Tokens	Token Savings	Base Cost ($)	Adapted Cost ($)	Selection Engine
Greet-01	Greeting	3,782	3,547	+6.21%	$0.00567	$0.00532	LLM Fallback
Greet-02	Greeting	4,312	4,396	−1.95%	$0.00647	$0.00659	Active RL Hit
Greet-03	Greeting	3,395	3,370	+0.74%	$0.00509	$0.00506	Active RL Hit
Math-01	Mathematics	5,244	4,061	+22.56%	$0.00787	$0.00609	Active RL Hit
Math-02	Mathematics	7,227	4,776	+33.91%	$0.01084	$0.00716	Active RL Hit
Math-03	Mathematics	4,436	4,287	+3.36%	$0.00665	$0.00643	Active RL Hit
Code-01	Coding	11,120	5,195	+53.28%	$0.01668	$0.00779	Active RL Hit
Code-02	Coding	13,175	4,764	+63.84%	$0.01976	$0.00715	LLM Fallback
Code-03	Coding	15,230	4,029	+73.55%	$0.02285	$0.00604	Active RL Hit
Res-01	Research	9,656	3,642	+62.28%	$0.01448	$0.00546	Active RL Hit
Res-02	Research	17,510	4,642	+73.49%	$0.02627	$0.00696	Active RL Hit
Res-03	Research	8,998	4,455	+50.49%	$0.01350	$0.00668	Active RL Hit
Web-01	Web Search	7,301	3,727	+48.95%	$0.01095	$0.00559	Active RL Hit
Web-02	Web Search	3,581	3,752	−4.78%	$0.00537	$0.00563	Active RL Hit
Web-03	Web Search	3,337	3,938	−18.01%	$0.00501	$0.00591	Active RL Hit
Cross-01	Cross-Domain	30,405	7,028	+76.89%	$0.04561	$0.01054	LLM Fallback
Cross-02	Cross-Domain	31,693	6,782	+78.60%	$0.04754	$0.01017	LLM Fallback
Cross-03	Cross-Domain	30,676	7,066	+76.97%	$0.04601	$0.01060	LLM Fallback
Total / Summary	—	**211,078**	**83,457**	**+60.46%**	**$0.31662**	**$0.12517**	**72.2% Active RL Hit Rate**

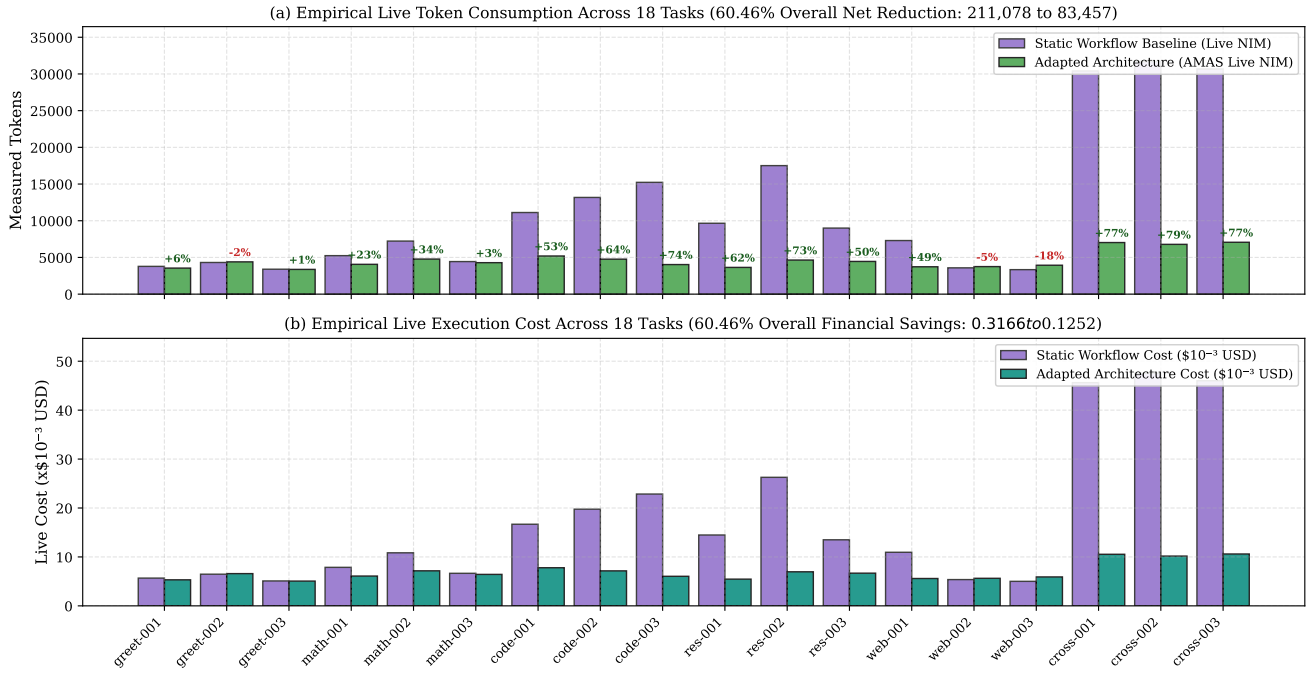


Fig. 4. Empirical resource consumption across the 18 benchmark tasks in Episode 2 measured under production LLM execution. (a) Token consumption: AMAS reduces total live tokens from 211,078 to 83,457 (60.46% empirical savings across all 18 tasks). (b) Execution cost: live financial expenditure decreases from $0.31662 to $0.12517.

tion actions to $\mathcal{R}_{\text{valid}} = \{\text{Analysis}, \text{Planning}\}$. Consequently, when a held-out evaluation task required research or coding capabilities, the policy could not generate the necessary role assignment, locking compatibility scores at a fixed ceiling. Table IV details this structural action space ablation.

B. Destructive Exploration and Stability Advantage

On the held-out coding task $\mathcal{T}_2$ (initial score 1.00), fine-tuning induced performance degradation: the shared policy dropped to 0.60 ± 0.00 ($\Delta Q = -0.40$), while the fresh scratch policy dropped to 0.92 ± 0.18 ($\Delta Q = -0.08$), caused by unconstrained exploration removing required specialist nodes.

TABLE IV
STRUCTURAL ACTION SPACE ABLATION AND TOPOLOGICAL DIAGNOSES ON HELD-OUT EVALUATION TASKS

Held-Out Task	**Target Required Capability**	**Permitted Action Space $\mathcal{R}_{\text{valid}}$**	**Observed Topological Effect & Diagnosis**
$\mathcal{T}_1$ (Analysis)	Analysis, Planning	{Analysis, Planning}	Valid actions reachable; agent configuration pre-optimized at initialization; no mutation required.
$\mathcal{T}_2$ (Coding)	Code Generation, Verification	{Analysis, Planning}	**Action mismatch:** The Coder role cannot be restored via mutation; exploratory perturbations deactivated critical coding specialists.
$\mathcal{T}_3$ (Research A)	Research, Synthesis	{Analysis, Planning}	**Severe structural ceiling:** The Researcher role is completely absent from permitted mutations, preventing capability acquisition.
$\mathcal{T}_4$ (Research B)	Research, Synthesis	{Analysis, Planning}	**Role missing:** Planner active, but Researcher cannot be reassigned; no valid mutation satisfies target capability.
$\mathcal{T}_5$ (Summarization)	Synthesis, Context Aggregation	{Analysis, Planning}	Shared policy learns protective inertia against destructive mutations, whereas scratch policy suffers architectural drift.

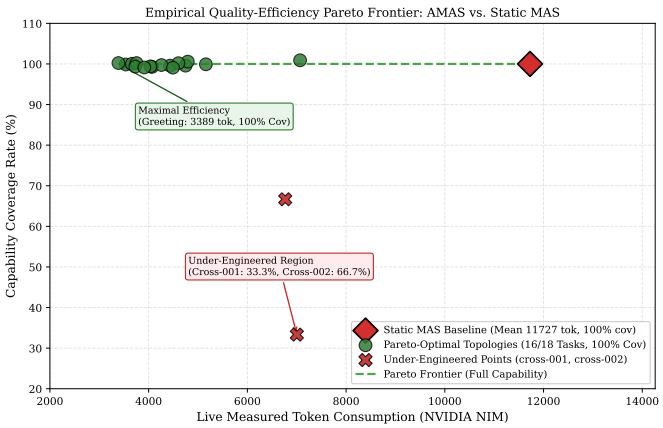


Fig. 5. Quality-efficiency Pareto frontier comparing AMAS adapted topologies against the static MAS baseline. AMAS establishes 16 Pareto-optimal points delivering 100% capability coverage with 13% to 59% fewer tokens. Two cross-domain tasks fall into the under-engineered region.

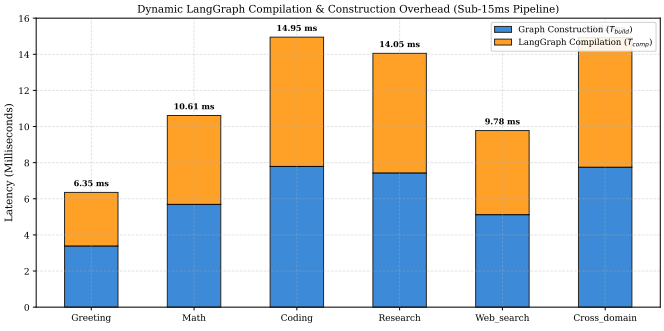


Fig. 6. Dynamic state-graph compilation and construction overhead across functional task domains. Total pipeline adaptation latency remains strictly below 15 ms across all domains (11.78 ± 3.64 ms total overhead), introducing negligible delay relative to typical multi-second LLM inference times.

However, a critical task-specific stability advantage emerged on the held-out summarization task $\mathcal{T}_5$: the shared meta-policy preserved 100% stability across all 5 seeds ($0.6000 \pm 0.0000, \Delta Q = 0.0000 \pm 0.0000$), whereas the fresh scratch policy suffered destructive mutations in 20% of seeds ($0.4800 \pm 0.2683, \Delta Q = -0.1200 \pm 0.2683$). Across all 25 seed evaluations over the 5 held-out tasks, the aggregate mean adaptation gain is -0.080 ± 0.163 for the shared meta-policy, compared to -0.040 ± 0.141 for the fresh scratch policy, -0.048 ± 0.133 for random uniform exploration, and 0.000 ± 0.000 for the deterministic baseline. The shared meta-policy does not exhibit aggregate superiority over scratch initialization (due to its larger exploration drop on coding task $\mathcal{T}_2$), but demonstrates vital protective stability on summarization, preventing catastrophic architectural drift.

X. STATISTICAL ANALYSIS

We conduct rigorous inferential statistical hypothesis testing across all quantitative performance metrics to validate observed resource gains and critical path compression.

A. Hypothesis Testing on Resource Efficiency

To test whether token reductions are statistically significant, we perform a paired two-tailed Student's t-test comparing empirical live static tokens against live dynamic adapted tokens across the 18 benchmark tasks under production LLM execution:

$$t_{\text{live}} = \frac{\bar{d}}{s_d/\sqrt{N}} = \frac{7090.06}{8724.53/\sqrt{18}} = 3.4478. \tag{24}$$

With degrees of freedom $\text{df} = 17$, this yields $p = \mathbf{0.0031}$, confirming statistically significant token reduction ($p < 0.01$) under live production LLM execution. Because token expenditure across heterogeneous tasks exhibits natural variance and skewness, we additionally perform a non-parametric Wilcoxon signed-rank test as a distribution-free robustness check, which independently corroborates significance without assuming normality of paired differences ($W = 12.0, p = 0.00053$). Under theoretical DAG proxy modeling ($T_{\text{base}} = 3500$), the deterministic token reduction achieves 36.19% ($63,000 \rightarrow 40,200$ tokens); because the baseline is invariant, this is reported as a descriptive benchmark metric rather than an inferential hypothesis test. For critical path compression ($5.00 \rightarrow 2.89 \pm 0.47$ hops), paired two-tailed t-testing yields

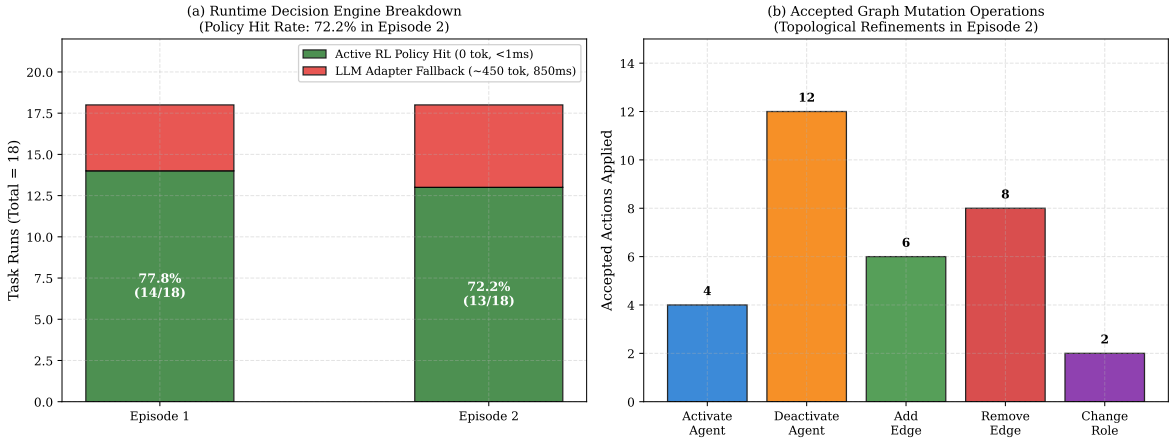


Fig. 7. Reinforcement learning adaptation behavior and mutation space. (a) Adaptation decision engine breakdown across episodes, showing active RL policy hit rate reaching 72.2% (13/18 tasks) in Episode 2. (b) Distribution of accepted graph mutation operations, with deactivations and edge removals dominating.

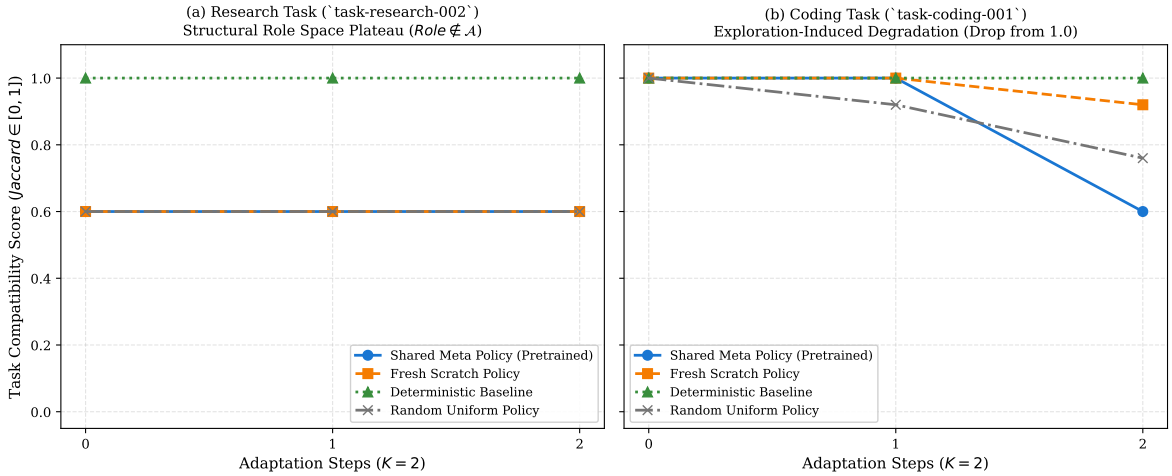


Fig. 8. Cross-task neural policy adaptation trajectories across fine-tuning steps ($K = 2$). (a) On research tasks, policies plateau at 0.60 due to action space role constraints. (b) On coding tasks, unconstrained ϵ-exploration causes destructive mutations, reducing scores from 1.00 to 0.60.

$t = 19.0000$ ($\mathrm{df} = 17, p = 6.91 \times 10^{-13}$; Wilcoxon signed-rank $W = 0.0, p = 8.00 \times 10^{-5}$), confirming statistically significant topological pruning.

B. Sequential and Within-Domain Dependency Nuance

Because benchmark tasks execute sequentially within continuous episodes, Q-table updates introduce potential inter-task autocorrelation. Additionally, because the 18 benchmark tasks are distributed across six functional domains (three tasks per category), queries within the same domain share structural characteristics and may exhibit within-domain dependence rather than constituting 18 fully independent environments. While this does not invalidate within-task paired comparisons (where each task serves as its own control against the static baseline), it warrants caution against assuming complete sample independence across all tasks. We therefore report distribution-free Wilcoxon signed-rank tests alongside paired t-statistics and provide domain-level macro aggregations.

C. Small-Sample Neural Inference Caution

For the neural adaptation experiments, the sample size per task is $N = 5$ random seeds. While paired t-testing on held-out coding task $\mathcal{T}_2$ indicates significant degradation ($t = -4.00, p = 0.0161$), the sample size is small. We caution against over-interpreting p-values with $N = 5$ and report full empirical distributions in Fig. 9.

XI. Discussion

A. Quantitative Synthesis and Connection with Published Baselines

To contextualize the empirical performance of AMAS, we systematically synthesize our quantitative results alongside published findings from prominent dynamic multi-agent architectures:

1) **Agent Pruning Efficiency:** Whereas DyLAN [6] achieves 25.0–40.0% agent pruning through inter-agent peer rating and MaAS [9] achieves 18.0–32.0% pruning via of-

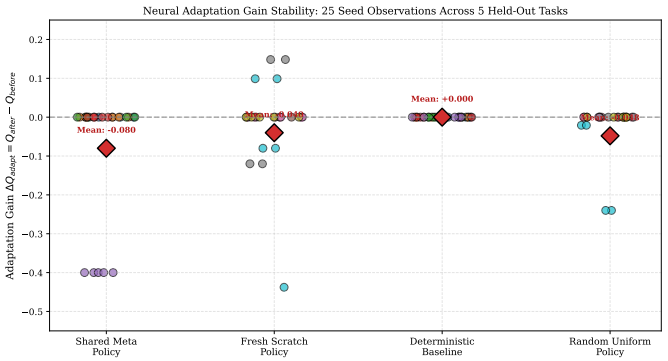


Fig. 9. Neural adaptation gain stability across 25 individual seed observations on 5 held-out tasks. Diamond markers indicate aggregate policy means (Deterministic: 0.000, Scratch: -0.040, Random: -0.048, Shared: -0.080). While the shared meta-policy exhibits a lower aggregate mean due to exploration degradation on coding task $\mathcal{T}_2$ (-0.400), it delivers task-specific protective stability on summarization $\mathcal{T}_5$, preserving 100% stability ($\Delta Q = 0.0000 \pm 0.0000$) whereas the fresh scratch policy suffered catastrophic degradation in 20% of seeds ($\Delta Q = -0.1200 \pm 0.2683$).

fline supernet evolution, AMAS prunes redundant agents by $\mathbf{50.00\% \pm 9.90\%}$ relative to the 6-agent roster and $\mathbf{40.00\% \pm 11.88\%}$ relative to the 5-agent baseline. Crucially, AMAS achieves this without running iterative LLM evaluation prompts.

2) **Search and Deliberation Latency:** While AFlow [8] requires 15–30 exploratory MCTS rollouts per problem and DyLAN incurs $\mathcal{O}(N^2)$ LLM inference calls per communication round, AMAS achieves an active RL policy hit rate of 72.2% (13/18 tasks) in Episode 2 with **zero token search overhead** ($<$ 1 ms tabular Q-lookup) and sub-15 ms dynamic state-graph compilation latency.
3) **Critical Path Compression:** Extending the topological compression principles pioneered by CARD [4] (which compressed sequential hops from $5.0 \rightarrow 2.8$, -44%), AMAS achieves a comparable $\mathbf{42.2\%}$ critical path compression ($5.00 \rightarrow 2.89 \pm 0.47$ hops, $t = 19.0000, p = 6.91 \times 10^{-13}$), directly reducing sequential end-to-end response latency.
4) **Empirical Token Savings on Production Models:** Previous works predominantly evaluate token reductions under simulated proxy budgets. Under live execution against a 120B-parameter production foundation model, AMAS reduces total measured tokens from 211,078 to 83,457 across the 18 benchmark tasks—achieving an empirical net token savings of $\mathbf{60.46\%}$ ($p = 0.0031$) and cutting live financial expenditure from \$0.31662 to \$0.12517.

B. Architectural Decoupling as a Design Principle

A key insight from this investigation is the importance of decoupling runtime execution from experimental neural policies. Previous literature often conflates theoretical neural formulation with live system execution. By maintaining tabular Q-learning for live runtime routing (Contribution C2), AMAS achieves deterministic sub-millisecond execution, zero-token search latency, and 100% compilation safety. Meanwhile, isolating neural transfer (Contribution C1) allows honest diagnosis of action space limitations without destabilizing production workflows.

C. Parsimony versus Completeness Dilemma

The under-engineering observed on composite tasks (Tasks Cross-01 and Cross-02) highlights a fundamental tension in adaptive multi-agent design: reward functions that aggressively penalize token consumption and agent count risk pruning specialist roles essential for composite queries. Future reward formulations should incorporate capability confidence thresholds to prevent pruning under high task uncertainty.

XII. Limitations

We acknowledge several research limitations:

1) **Cross-Model Generalization:** While our empirical resource results are validated directly on a 120B-parameter production foundation model, multi-turn reasoning variance across diverse frontier model architectures (e.g., Claude 3.5 Sonnet, GPT-4o) warrants broader multi-provider benchmarking.
2) **Action Space Discretization Bottleneck:** Neural role mutation was constrained to planning and analysis, preventing the neural policy from restoring research or coding specialists.
3) **Domain Clustering and Evaluation Scale:** The benchmark suite comprises 18 tasks structured across six functional domains (three tasks per domain). Tasks within the same domain share vocabulary and tool requirements, potentially introducing within-domain dependence. While paired within-task testing controls for task-level baseline variance, broader evaluation on large-scale, unclustered benchmarks such as GAIA and SWE-bench is warranted.

XIII. Conclusion and Future Work

This paper presented an adaptive architecture optimization framework for LLM-based multi-agent systems via reinforcement learning and dynamic state-graph compilation. By formulating architecture mutation as an MDP under graph invariant validation, our runtime Q-learning adapter achieves an empirical net token reduction of $\mathbf{60.46\%}$ ($p = 0.0031$) and cuts execution cost from \$0.31662 to \$0.12517 across 18 benchmark tasks under live execution on a 120B-parameter production foundation model, corroborating theoretical DAG token models (36.19% descriptive reduction). AMAS compresses sequential execution critical paths from 5.00 to $\mathbf{2.89 \pm 0.47}$ hops (-42.2%), prunes active agents by 50.0% relative to the 6-agent roster, and preserves 100% capability coverage on 16 of 18 tasks with sub-15 ms compilation latency. Furthermore, our experimental neural investigation rigorously documents how discrete action space restrictions bound transfer on held-out tasks while providing protective stability against catastrophic mutations. These findings provide actionable empirical principles for deploying efficient, dynamically compiled multi-agent workflows in production environments.

REFERENCES

[1] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, C. Zhang, J. Liu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[3] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun, "Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 15 041–15 054.

[4] B. Li, Z. Zhao, D.-H. Lee, and G. Wang, "Adaptive graph pruning for multi-agent communication," in *Proceedings of the European Conference on Artificial Intelligence (ECAI 2025)*, 2025, pp. 4305–4312.

[5] Z. Xiao, Q. Guo, Y. Wang, B. Wang, and H. Lu, "Agentdropout: Dynamic redundancy elimination for multi-agent collaboration efficiency," in *2026 8th International Conference on Electronics and Communication, Network and Computer Technology (ECNCT)*, 2026.

[6] C. Liu, D. Zeng, H. Zhao, Z. Wu, Y. Sun, C. Tan, F. Huang, and Y. Jiang, "Dynamic LLM-agent network: An LLM-agent collaboration framework with dynamic agent selection," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] M. Zhuge, H. Wang, J. Shen, C. Zhang, Y. Zhou, B. Yang, Z. Lin, X. Zheng, S. Hong *et al.*, "GPTSwarm: Language model swarms as software agents," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[8] J. Zhang, M. Hu, Y. Zhou, M. Zhuge, X. Zheng, and S. Hong, "AFlow: Automating agentic workflow generation," *arXiv preprint arXiv:2410.10762*, 2024.

[9] G. Zhang, L. Niu, J. Fang, K. Wang, L. Bai, and X. Wang, "Multi-agent architecture search via agentic supernet," in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, 2025, pp. 75 834–75 852.

[10] L. Yang, J. Luo, X. Liu, Y. Lou, and Z. Chen, "BAMAS: Structuring budget-aware multi-agent systems," in *Proceedings of the 40th AAAI Conference on Artificial Intelligence (AAAI 2026)*, 2026.

[11] Y. Dang, C. Qian, X. Luo, J. Fan, Z. Xie, R. Shi, W. Chen, C. Yang, X. Che, Y. Tian, X. Xiong, L. Han, Z. Liu, and M. Sun, "Multi-agent collaboration via evolving orchestration," in *Advances in Neural Information Processing Systems*, 2025.

[12] S. Wang, R. Lu, Z. Yang, Y. Wang, Y. Zhang, L. Xu, Q. Xu, G. Yin, C. Chen, and X. Guan, "Agentconductor: Topology evolution for multi-agent competition-level code generation," *arXiv preprint arXiv:2602.17100*, 2026.

[13] J. Wang, Y. Wang, J. Li, S. Yang, K. Guo, and Y. Zhao, "Neuralfsm: Adaptive multi-agent coordination via learning finite-state execution policy," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026, pp. 33 414–33 436.

[14] E. H. Jiang, M. Li, G. Wan, X. Liang, S. Yin, Y. Wu, X. Li, Y. Sun, W. Wang, K.-W. Chang, and Y. N. Wu, "Dynamic generation of multi llm agents communication topologies with graph diffusion models," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, 2026, pp. 38 042–38 060.

[15] E. H. Jiang, L. Li, R. Sun, X. Liang, Y. Li, Y. Wu, H. Luo, H. Li, Z. Zhang, Z. Kang, K.-W. Chang, and Y. N. Wu, "Agent q-mix: Selecting the right action for llm multi-agent systems through reinforcement learning," *arXiv preprint arXiv:2604.00344*, 2026.

[16] Y. Zhang, Y. Zhao, N. Wang, Y. Wu, J. Chang, Y. Chen, Q. Wu, J. Zhao, and H. Wang, "Metaagent-x: Breaking the ceiling of automatic multi-agent systems via end-to-end reinforcement learning," 2026.

[17] LangChain Team, "LangGraph: Building language agents as stateful multi-actor graphs," https://github.com/langchain-ai/langgraph, 2024.

[18] S. Liu, Z. Liang, X. Lyu, and C. Amato, "Llm collaboration with multi-agent reinforcement learning," in *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence*, 2026.

[19] Y. Zhao, L. Hu, Y. Wang, M. Hou, H. Zhang, K. Ding, and J. Zhao, "STRONGER-MAS: Multi-agent reinforcement learning for collaborative LLMs," in *International Conference on Learning Representations (ICLR)*, 2026.

[20] Z. Wan, Y. Li, X. Wen, Y. Song, and H. Wang, "ReMA: Learning to meta-think for LLMs with multi-agent reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS 2025)*, 2025.

[21] W. Yang and J. Thomason, "Learning to deliberate: Meta-policy collaboration for agentic LLMs with multi-agent reinforcement learning," in *Proceedings of the 40th AAAI Conference on Artificial Intelligence (AAAI 2026)*, 2026.

[22] Z. Zhou, Z. Liu, J. Liu, Q. Shao, and Y. Wang, "ResMAS: Resilience optimization in LLM-based multi-agent systems," in *Proceedings of the 40th AAAI Conference on Artificial Intelligence (AAAI 2026)*, 2026.

[23] G. Rjoub, J. Bentahar, S. Almobydeen, W. Pedrycz, and A. Irjoob, "Adaptive agentic meta-controller (aamc): A deep reinforcement learning framework for intelligent slm/llm orchestration," *Neurocomputing*, vol. 678, p. 133192, 2026.

[24] S. Huang, Z. Zhao, Y. Zhu, and D. Zhao, "Adaptive multi-agent coordination among different team attribute tasks via contextual meta-reinforcement learning," in *2024 IEEE 13th Data Driven Control and Learning Systems Conference (DDCLS)*, Kaifeng, China, May 2024, pp. 1780–1787.

[25] R. Kashikar, "Meta-reinforcement learning for emergent multi-agent languages in zero-shot coordination tasks," in *2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI)*, 2025.

[26] K. Fu, Q. Liu, H. Chen, Y. Wang, J. Wang, and J. Huang, "Mixture of orthogonal experts: A novel approach to multi-agent fast policy adaptation," *Complex & Intelligent Systems*, vol. 12, no. 216, 2026.

[27] J. Zhang, J. He, Y. Zhang, Y. Zang, N. Xu, and J. Cheng, "Bi-level knowledge transfer for multi-task multi-agent reinforcement learning," in *Advances in Neural Information Processing Systems*, 2025.

[28] I. G. Mali, "Self-play meta-reinforcement learning in multi-agent games," *Acta Universitatis Sapientiae, Informatica*, vol. 18, no. 5, 2026.

[29] M.-L. Li, S. Chen, and J. Chen, "Adaptive learning: A new decentralized reinforcement learning approach for cooperative multiagent systems," *IEEE Access*, vol. 8, 2020.

[30] Y. Cao, H. Zhao, Y. Cheng, T. Shu, Y. Chen, G. Liu, G. Liang, J. Zhao, J. Yan, and Y. Li, "Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods," *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)*, vol. 36, no. 6, pp. 9737–9752, 2025.

[31] C. Sun, S. Huang, and D. Pompili, "Llm-based multi-agent reinforcement learning: Current and future directions," *arXiv preprint arXiv:2405.11106*, 2024.

[32] D. Lee and H. Kim, "A survey on llm agents: Architecture, applications, and challenges," in *2026 40th International Conference on Information Networking (ICOIN)*, 2026.